Q7 - Diagnose, Redesign and Validate

C Programming | Day-4 Design Challenge | Bloom's Level K5 - Evaluate | 13 Marks

Student: SEETHARAMAN S Reg. No: 192313051A

Selected program: Prime-number test using a loop (from the loop design / termination task, Q6).

Original program (prime_original.c)

```
#include <stdio.h>

/* ORIGINAL: trial division by every i from 2 to n-1 */
int is_prime_original(long n, long *divs, long *iters) {
    if (n < 2) return 0;
    for (long i = 2; i < n; i++) {
        (*iters)++;
        (*divs)++;                 /* one modulo (division) per iteration */
        if (n % i == 0) return 0;
    }
    return 1;
}

int main(void) {
    long inputs[] = {97, 997, 9973, 99991, 999983, 9999991};
    printf("n,prime,divisions,iterations\n");
    for (int k = 0; k < 6; k++) {
        long divs = 0, iters = 0;
        int p = is_prime_original(inputs[k], &divs, &iters);
        printf("%ld,%d,%ld,%ld\n", inputs[k], p, divs, iters);
    }
    return 0;
}
```

A. Diagnose the Bottleneck (2 Marks)

The original program divides n by **every** integer i from 2 to n-1. Almost all work is the single statement `n % i == 0` inside the loop: one division per iteration, so divisions = iterations. The raw counts show the count grows linearly with the input: for prime n the program performs exactly n-2 divisions (97 -> 95, 997 -> 995, 9,999,991 -> 9,999,989); every 10x increase in n gives a 10x increase in divisions.

Work is unnecessarily repeated in two ways: (1) **even divisors** 4, 6, 8, ... are tested although n has already failed the test for 2 (if n is not divisible by 2 it cannot be divisible by any even number); (2) **divisors above sqrt(n)** are tested although every composite n must have a factor <= sqrt(n); a factor larger than sqrt(n) always pairs with a smaller one that would already have been found. For a prime the loop therefore runs to completion, doing about 99.98% wasted work at n = 10^7.

B. Define the Design Objective (2 Marks)

Objective: Reduce the number of divisions for a prime input from O(n) to O(sqrt(n)), specifically to no more than (sqrt(n)/2) + 1, i.e. skip all even divisors and all divisors greater than sqrt(n); keep the result identical for every n (correctness preserved) and guarantee termination (loop variable strictly increases by 2 and i*i exceeds n in finite steps).

C. Design the Revised Solution (4 Marks)

What changed: (1) handle n < 2, n = 2 and even n separately with one test; (2) loop only over odd i starting at 3 with step 2; (3) change the loop condition from i < n to i*i <= n. **Property exploited:** if n = a*b with a <= b then a <= sqrt(n); and every even composite is divisible by 2. **Extra memory:** none (O(1)); no table is required. **New trade-offs:** one extra comparison/branch for the special cases, a multiplication i*i per iteration (cheaper than the division it saves), slightly more complex code, and a potential overflow of i*i for very large n (long is used; for n near 2^63 a check i <= n/i would be needed). For n = 2 the redesign

performs 1 division while the original performs 0.

Pseudocode

```
FUNCTION isPrime(n)
    IF n < 2 THEN RETURN false
    divisions <- divisions + 1
    IF n == 2 THEN RETURN true
    IF n mod 2 == 0 THEN RETURN false
    i <- 3
    WHILE i * i <= n DO
        divisions <- divisions + 1
        IF n mod i == 0 THEN RETURN false
        i <- i + 2
    END WHILE
    RETURN true
END FUNCTION
```

Flowchart (text form)

```
[Start] -> [Read n] -> <n < 2?> --yes--> [Not prime]
                            | no
                       <n == 2?> --yes--> [Prime]
                            | no
                     <n mod 2 == 0?> --yes--> [Not prime]
                            | no
                          [i = 3]
                            |
                  +-> <i*i <= n?> --no--> [Prime]
                  |         | yes
                  |    <n mod i == 0?> --yes--> [Not prime]
                  |         | no
                  +---- [i = i + 2]
```

D. Predict Before Implementing (1 Mark)

Prediction: The number of divisions for a prime n will fall from about n-2 to about sqrt(n)/2. For n = 9,999,991 (sqrt ~ 3162) I predict about 1,581 divisions instead of 9,999,989, a reduction of roughly 6,300x, measured in **division count**. For composite even n and for n with a tiny smallest factor, the count should stay about the same (1-2 divisions), and results must match the original for all n.

E. Implement and Validate the Redesign (2 Marks)

Both programs were compiled with gcc and run on the same six prime inputs. Raw output (CSV: n, prime?, divisions, iterations):

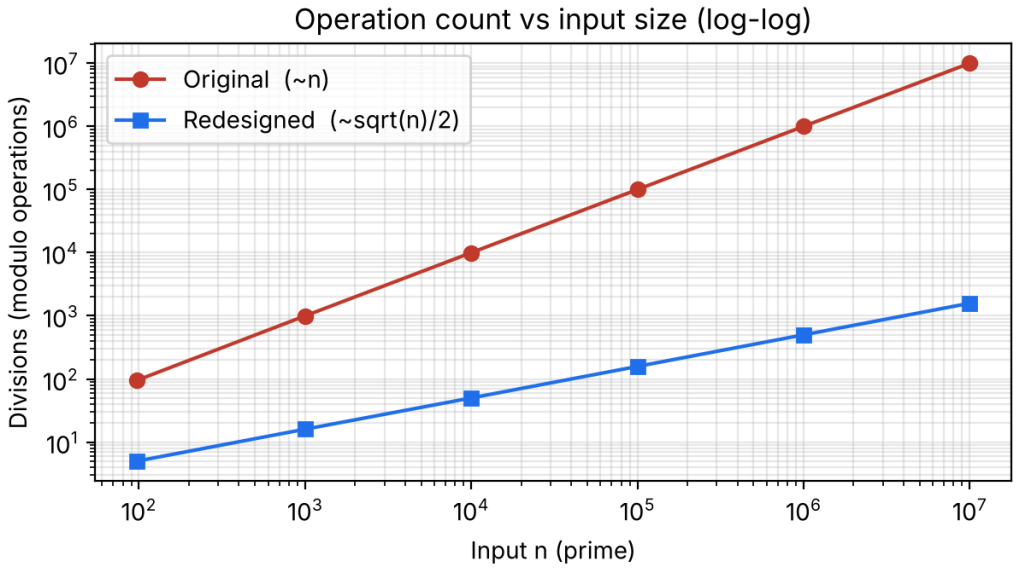
```
#include <stdio.h>

/* REDESIGNED: test 2 once, then only odd divisors; stop when i*i > n.
   Exploits: a composite n has a factor <= sqrt(n);
   every even n > 2 is caught by the single test for 2. */
int is_prime_redesigned(long n, long *divs, long *iters) {
    if (n < 2) return 0;
    (*divs)++;
    if (n == 2) return 1;
    if (n % 2 == 0) return 0;
    for (long i = 3; i * i <= n; i += 2) {
        (*iters)++;
        (*divs)++;
        if (n % i == 0) return 0;
    }
    return 1;
}

int main(void) {
    long inputs[] = {97, 997, 9973, 99991, 999983, 9999991};
    printf("n,prime,divisions,iterations\n");
    for (int k = 0; k < 6; k++) {
        long divs = 0, iters = 0;
        int p = is_prime_redesigned(inputs[k], &divs, &iters);
        printf("%ld,%d,%ld,%ld\n", inputs[k], p, divs, iters);
    }
```

```
        return 0;
    }
```

n	Original divisions	Redesigned divisions	Reduction (x)	Predicted sqrt(n)/2
97	95	5	19	5
997	995	16	62	16
9,973	9,971	50	199	50
99,991	99,989	158	633	158
999,983	999,981	500	2,000	500
9,999,991	9,999,989	1,581	6,325	1,581



Edge cases and composites (same inputs for both versions):

n	Original: prime?	Orig. div	Redesign: prime?	Redesign div
2	yes	0	yes	1
3	yes	1	yes	1
4	no	1	no	1
9	no	2	no	2
1,000,000	no	1	no	1
9,999,999	no	2	no	2
1	no	0	no	0
9,999,973	yes	9999971	yes	1581

Correctness check: the two functions were compared for every n from 0 to 20,000: **0 mismatches**. Results are identical on all test inputs, so the redesign is validated.

F. Defend the Final Design (2 Marks)

Objective achieved: Yes. For the largest prime tested (9,999,991) divisions fell from 9,999,989 to 1,581 (about 6,325x), matching the predicted sqrt(n)/2 (= 1,581). The reduction grows with n: 19x at n = 97, 62x at 997, 199x at 9,973, 633x at 99,991, 2,000x at 999,983. This confirms the complexity change from O(n) to O(sqrt(n)), not merely a constant-factor gain. Correctness was preserved (0 mismatches).

New costs: an extra branch for the special cases, an i*i multiplication every iteration, a risk of overflow for n close to the type limit, and slightly harder-to-read code. No extra memory is used.

Where the original may still be preferable: for very small inputs and for composite numbers with a small factor, both versions stop after 1-2 divisions, so the redesign gives no gain (n = 4, 9, 1,000,000, 9,999,999 gave identical counts). For n = 2 the redesign is actually one division worse (1 vs 0). If the code only ever tests tiny numbers, the original is simpler and easier to verify by inspection. Further improvements (6k+-1 wheel, a sieve table for repeated queries at the cost of O(N) memory) exist but were outside the stated objective.

Conclusion: The redesign is defensible for general use because worst-case work (primes) is reduced from linear to square-root growth with no memory cost and verified identical output.

Submission Evidence Checklist

C source for both versions (included above); raw counts (tables above and CSV output); graph (above). Add your GitHub repository link containing the code, input files, output screenshots and README.md.